T3000 API & Protocol

Device Troubleshooting · built Nov 13, 2026

Device Troubleshooting

Common issues and solutions for T3000 device connectivity and operation.

Connection Issues

Device Not Discovered

Symptoms:

- Device not appearing in discovery scan
- "No devices found" message

Solutions:

1. **Check Network Connectivity**

```shell
# Ping device IP
ping 192.168.1.100

# Check port availability
telnet 192.168.1.100 47808
```

2. **Verify Device Settings**
   - Confirm device IP address is correct
   - Check subnet mask matches network
   - Verify gateway configuration
   - Ensure BACnet port is 47808 (UDP)
3. **Check Firewall**
   - Allow UDP port 47808 (BACnet/IP)
   - Allow TCP port 502 (Modbus TCP)
   - Disable antivirus temporarily to test

- Check Windows Firewall rules

4. **Network Broadcast Issues**
   - Some networks block broadcast packets
   - Try manual device addition instead
   - Contact IT to enable multicast

Connection Timeout

Symptoms:

- "Connection timeout" error
- Slow response times
- Intermittent disconnections

Solutions:

1. **Increase Timeout Values**
   - Go to Settings > Communication
   - Increase timeout from 5s to 10s
   - Increase retry count
2. **Reduce Network Load**
   - Decrease polling frequency
   - Limit concurrent connections
   - Close other network applications
3. **Check Network Quality**
   - Test network latency: `ping -n 100 192.168.1.100`
   - Check for packet loss
   - Verify network switch/router health

Authentication Failed

Symptoms:

- "Authentication error" message
- "Access denied" errors

Solutions:

1. **Verify Credentials**

- Check username/password
- Verify case sensitivity
- Try default credentials (if applicable)

2. **Check User Permissions**
   - Ensure user has adequate permissions
   - Verify user account is not locked
   - Contact administrator for access

3. **Device Configuration**
   - Verify authentication is enabled
   - Check allowed IP addresses
   - Review security settings

Data Reading Issues

Points Not Updating

Symptoms:

- Data values frozen/stale
- "Last update" timestamp old
- No new trend data

Solutions:

1. **Check Connection Status**
   - Verify device is online (green indicator)
   - Test communication with manual refresh
   - Review connection logs

2. **Verify Auto-Refresh**
   - Ensure auto-refresh is enabled
   - Check refresh interval setting
   - Try manual refresh (🔄 button)

3. **Clear Cache**

```javascript
// Browser console
localStorage.clear();
location.reload();
```

4. **Restart Services**
   - Restart T3000 API server
   - Refresh browser page
   - Reconnect device

Incorrect Values

Symptoms:

- Values don't match physical readings
- Negative values where impossible
- Out-of-range readings

Solutions:

1. **Check Calibration**
   - Review input calibration settings
   - Verify sensor type configuration
   - Check units conversion
2. **Verify Point Configuration**
   - Confirm correct input type (0-10V, 4-20mA)
   - Check scaling factors
   - Verify offset settings
3. **Test Hardware**
   - Check sensor wiring
   - Test with multimeter
   - Replace faulty sensors

Performance Issues

Slow Page Loading

Symptoms:

- Pages take long to load
- Browser freezes temporarily
- High CPU usage

Solutions:

1. **Reduce Data Load**
    - Decrease number of visible points
    - Increase refresh interval
    - Limit trend chart points
2. **Browser Optimization**
    - Use latest browser version
    - Clear browser cache
    - Close unused tabs
    - Disable browser extensions
3. **System Resources**
    - Close other applications
    - Check available RAM
    - Monitor CPU usage

Database Errors

Symptoms:

- "Database locked" errors
- "Failed after X retries" messages
- Slow save operations

Solutions:

1. **Wait and Retry**
    - Database automatically retries with exponential backoff
    - Wait up to 51 seconds for retry completion
    - Operation should succeed eventually
2. **Check Server Load**
    - Monitor API server CPU/memory
    - Reduce concurrent operations

- Close other connections

3. **Database Maintenance**
   - Vacuum database (if SQLite)
   - Archive old trend data
   - Optimize database indexes

Trend Log Issues

No Trend Data

Symptoms:

- Empty trend charts
- "No data available" message
- Missing historical data

Solutions:

1. **Verify Trend Logging Enabled**
   - Check point configuration
   - Enable trend logging
   - Set appropriate log interval
2. **Check Time Range**
   - Verify selected time range has data
   - Try different date range
   - Check data retention settings
3. **Database Storage**
   - Verify sufficient disk space
   - Check database permissions
   - Review database logs

Gaps in Trend Data

Symptoms:

- Missing data segments
- Discontinuous charts
- Irregular timestamps

Causes:

- Network interruptions
- Device offline periods
- Database errors
- Polling failures

Solutions:

- Review connection logs for outages
- Check alarm history
- Verify continuous power supply

Alarm Issues

Alarms Not Triggering

Symptoms:

- No alarms despite out-of-range values
- Silent alarm conditions

Solutions:

1. **Verify Alarm Configuration**
    - Check alarm enable status
    - Verify threshold values
    - Review alarm priority
2. **Check Notification Settings**
    - Verify notification channels enabled
    - Test email/SMS configuration
    - Check browser notification permissions

False Alarms

Symptoms:

- Alarms for normal conditions
- Nuisance alarms

Solutions:

1. **Adjust Thresholds**

- Review alarm setpoints
- Add deadband/hysteresis
- Use time delays

2. **Filter Noise**
   - Increase sensor filter time
   - Use averaging
   - Check for electrical interference

Browser Compatibility

UI Elements Not Working

Supported Browsers:

- ✅ Chrome 90+
- ✅ Edge 90+
- ✅ Firefox 88+
- ✅ Safari 14+

Not Supported:

- ❌ Internet Explorer
- ❌ Old browser versions

Solutions:

1. Update to latest browser version
2. Clear browser cache
3. Disable conflicting extensions
4. Try incognito/private mode

API Server Issues

Server Not Running

Check Server Status:

```powershell
## Windows
Get-Process | Where-Object { $_.Path -like "*T3000*api*" }

## If not running, start server
cd api
cargo run --release
```

Solutions:

1. Ensure API server is running on port 9103
2. Check server logs for errors
3. Verify no port conflicts
4. Restart server if needed

API Errors

Common Error Codes:

- **404**: Endpoint not found (check API path)
- **500**: Internal server error (check logs)
- **503**: Service unavailable (server overloaded)
- **timeout**: Server not responding

Getting Help

If issues persist:

1. **Check Logs**
   - Browser console (F12)
   - API server logs
   - Device logs
2. **Collect Information**
   - Error messages (exact text)
   - Steps to reproduce
   - Browser/OS version
   - Device firmware version
3. **Contact Support**

- Include all collected information
- Attach relevant logs
- Provide screenshots

Next Steps

- Best Practices - Preventive measures
- Performance Tuning - Optimization tips
- FAQ - Frequently asked questions

Device Troubleshooting

Diagnostic Tools

Network Diagnostics

```shell
## Test BACnet connectivity
nc -u -v 192.168.1.100 47808

## Send BACnet WhoIs packet
echo -ne '\x81\x0a\x00\x11\x01\x20\xff\xff\x00\xff\x10\x08' | nc -u 192.168.1.100 47808

## Test Modbus TCP
nc -v 192.168.1.100 502

## Capture BACnet traffic
tcpdump -i eth0 -n udp port 47808 -w bacnet.pcap
```

Connection Testing Script

TYPESCRIPT

```typescript
async function diagnostIcsCheck(deviceIp: string): Promise<DiagnosticReport> {
  const report: DiagnosticReport = {
    timestamp: new Date(),
    tests: []
  };

  // 1. Ping test
  const pingResult = await testPing(deviceIp);
  report.tests.push({
    name: 'ICMP Ping',
    passed: pingResult.reachable,
    latency: pingResult.avgLatency,
    details: `${pingResult.packetLoss}% packet loss`
  });

  // 2. BACnet port test
  const bacnetResult = await testPort(deviceIp, 47808, 'udp');
  report.tests.push({
    name: 'BACnet Port (47808/UDP)',
    passed: bacnetResult.open,
    details: bacnetResult.message
  });

  // 3. Modbus port test
  const modbusResult = await testPort(deviceIp, 502, 'tcp');
  report.tests.push({
    name: 'Modbus Port (502/TCP)',
    passed: modbusResult.open,
    details: modbusResult.message
  });

  // 4. BACnet WhoIs test
  const whoIsResult = await sendBACnetWhoIs(deviceIp);
  report.tests.push({
    name: 'BACnet WhoIs Response',
    passed: whoIsResult.responded,
    details: `Device Instance: ${whoIsResult.deviceInstance || 'N/A'}`
  });
```

```
  return report;
}
```

Log Analysis

TYPESCRIPT

```typescript
class LogAnalyzer {
  analyzeConnectionLogs(logs: LogEntry[]): ConnectionAnalysis {
    const errors = logs.filter(l => l.level === 'error');
    const warnings = logs.filter(l => l.level === 'warn');

    // Detect patterns
    const timeoutErrors = errors.filter(e =>
      e.message.includes('timeout') || e.message.includes('ETIMEDOUT')
    );

    const authErrors = errors.filter(e =>
      e.message.includes('auth') || e.message.includes('permission')
    );

    const networkErrors = errors.filter(e =>
      e.message.includes('ECONNREFUSED') || e.message.includes('ENETUNREACH')
    );

    return {
      totalErrors: errors.length,
      totalWarnings: warnings.length,
      commonIssues: {
        timeouts: timeoutErrors.length,
        authentication: authErrors.length,
        network: networkErrors.length
      },
      recommendations: this.generateRecommendations({
        timeoutErrors,
        authErrors,
        networkErrors
      })
    };
  }

  private generateRecommendations(issues: any): string[] {
    const recs: string[] = [];

    if (issues.timeoutErrors.length > 5) {
```

```
      recs.push('Increase timeout settings or reduce polling frequency');
    }

    if (issues.authErrors.length > 0) {
      recs.push('Verify device credentials and user permissions');
    }

    if (issues.networkErrors.length > 3) {
      recs.push('Check network connectivity and firewall settings');
    }

    return recs;
  }
}
```

Performance Debugging

Database Lock Investigation

TYPESCRIPT

```typescript
// Enable WAL mode to reduce locking
import Database from 'better-sqlite3';

const db = new Database('t3000.db');

// Check current journal mode
const journalMode = db.pragma('journal_mode', { simple: true });
console.log('Journal mode:', journalMode);

// Enable WAL for concurrent access
db.pragma('journal_mode = WAL');

// Monitor lock waits
db.function('log_lock_wait', (duration: number) => {
  console.warn(`Database lock waited ${duration}ms`);
});

// Set busy timeout
db.pragma('busy_timeout = 5000');
```

Query Performance Analysis

SQL

```sql
-- Enable query planner
EXPLAIN QUERY PLAN
SELECT * FROM trend_data
WHERE device_id = 389001
  AND point_id = 'IN1'
  AND timestamp BETWEEN 1704067200 AND 1704153600
ORDER BY timestamp DESC;

-- Analyze table statistics
ANALYZE trend_data;

-- Check index usage
SELECT * FROM sqlite_stat1 WHERE tbl = 'trend_data';

-- Find missing indexes
SELECT * FROM (
  SELECT 'Missing index on trend_data(device_id)' as suggestion
  WHERE NOT EXISTS (
    SELECT 1 FROM sqlite_master
    WHERE type = 'index'
    AND tbl_name = 'trend_data'
    AND sql LIKE '%device_id%'
  )
);
```

Memory Profiling

TYPESCRIPT

```typescript
class MemoryProfiler {
  private baseline: number = 0;

  start() {
    if (global.gc) global.gc();
    this.baseline = process.memoryUsage().heapUsed;
  }

  checkpoint(label: string) {
    const current = process.memoryUsage();
    const delta = current.heapUsed - this.baseline;

    console.log(`[${label}] Memory delta: ${(delta / 1024 / 1024).toFixed(2)} MB`);
    console.log(`  Heap Used: ${(current.heapUsed / 1024 / 1024).toFixed(2)} MB`);
    console.log(`  External: ${(current.external / 1024 / 1024).toFixed(2)} MB`);
  }

  detectLeaks(iterations: number = 100) {
    const samples: number[] = [];

    for (let i = 0; i < iterations; i++) {
      if (global.gc) global.gc();
      samples.push(process.memoryUsage().heapUsed);
    }

    // Check for upward trend
    const slope = this.calculateSlope(samples);
    if (slope > 1024 * 100) { // 100 KB per iteration
      console.warn('Potential memory leak detected!');
      console.warn(`Growth rate: ${(slope / 1024).toFixed(2)} KB per iteration`);
    }
  }

  private calculateSlope(data: number[]): number {
    const n = data.length;
    const sumX = (n * (n - 1)) / 2;
    const sumY = data.reduce((a, b) => a + b, 0);
    const sumXY = data.reduce((sum, y, x) => sum + x * y, 0);
```

```
    const sumX2 = (n * (n - 1) * (2 * n - 1)) / 6;

    return (n * sumXY - sumX * sumY) / (n * sumX2 - sumX * sumX);
  }
}
```

Automated Error Recovery

Retry Mechanism with Exponential Backoff

TYPESCRIPT

```typescript
async function withRetry<T>(
  fn: () => Promise<T>,
  options: {
    maxRetries?: number;
    baseDelay?: number;
    maxDelay?: number;
    onRetry?: (attempt: number, error: Error) => void;
  } = {}
): Promise<T> {
  const {
    maxRetries = 3,
    baseDelay = 1000,
    maxDelay = 30000,
    onRetry = () => {}
  } = options;

  let lastError: Error;

  for (let attempt = 0; attempt < maxRetries; attempt++) {
    try {
      return await fn();
    } catch (error) {
      lastError = error as Error;

      if (attempt < maxRetries - 1) {
        const delay = Math.min(
          baseDelay * Math.pow(2, attempt),
          maxDelay
        );

        onRetry(attempt + 1, error as Error);
        await sleep(delay);
      }
    }
  }
```

```
  throw new Error(`Failed after ${maxRetries} attempts: ${lastError.message}`);
}
```

Circuit Breaker Pattern

TYPESCRIPT

```typescript
class CircuitBreaker {
  private failures = 0;
  private lastFailureTime = 0;
  private state: 'closed' | 'open' | 'half-open' = 'closed';

  constructor(
    private threshold: number = 5,
    private timeout: number = 60000,
    private resetTimeout: number = 30000
  ) {}

  async execute<T>(fn: () => Promise<T>): Promise<T> {
    if (this.state === 'open') {
      if (Date.now() - this.lastFailureTime > this.resetTimeout) {
        this.state = 'half-open';
      } else {
        throw new Error('Circuit breaker is OPEN');
      }
    }

    try {
      const result = await fn();
      this.onSuccess();
      return result;
    } catch (error) {
      this.onFailure();
      throw error;
    }
  }

  private onSuccess() {
    this.failures = 0;
    this.state = 'closed';
  }

  private onFailure() {
    this.failures++;
    this.lastFailureTime = Date.now();
```

```
    if (this.failures >= this.threshold) {
      this.state = 'open';
      console.warn('Circuit breaker opened due to failures');
    }
  }
}
```

Self-Healing Connection Manager

TYPESCRIPT

```typescript
class SelfHealingConnectionManager {
  private connections = new Map<number, DeviceConnection>();
  private healthChecks = new Map<number, NodeJS.Timeout>();

  async ensureConnection(deviceId: number): Promise<DeviceConnection> {
    let conn = this.connections.get(deviceId);

    if (!conn || !conn.isHealthy()) {
      // Reconnect
      try {
        conn = await this.reconnect(deviceId);
        this.connections.set(deviceId, conn);
        this.startHealthCheck(deviceId);
      } catch (error) {
        console.error(`Failed to connect to device ${deviceId}:`, error);
        throw error;
      }
    }

    return conn;
  }

  private async reconnect(deviceId: number): Promise<DeviceConnection> {
    const device = await this.getDeviceConfig(deviceId);

    return await withRetry(
      () => this.createConnection(device),
      {
        maxRetries: 3,
        baseDelay: 2000,
        onRetry: (attempt, error) => {
          console.log(`Reconnecting to device ${deviceId}, attempt ${attempt}`);
        }
      }
    );
  }

  private startHealthCheck(deviceId: number) {
```

```
    const interval = setInterval(async () => {
      const conn = this.connections.get(deviceId);
      if (!conn) return;

      try {
        await conn.ping();
      } catch (error) {
        console.warn(`Health check failed for device ${deviceId}`);
        await this.reconnect(deviceId);
      }
    }, 30000); // Every 30 seconds

    this.healthChecks.set(deviceId, interval);
  }
}
```

Debugging Tools

Packet Capture Analysis

SHELL

```
## Capture BACnet traffic
tcpdump -i any -n udp port 47808 -w bacnet.pcap

## Analyze with tshark
tshark -r bacnet.pcap -Y bacnet -T fields \
  -e frame.time -e ip.src -e ip.dst -e bacnet.type

## Decode BACnet packet
tshark -r bacnet.pcap -Y bacnet -V | less
```

Debug Logging Configuration

TYPESCRIPT

```typescript
import winston from 'winston';

const logger = winston.createLogger({
  level: process.env.LOG_LEVEL || 'info',
  format: winston.format.combine(
    winston.format.timestamp(),
    winston.format.errors({ stack: true }),
    winston.format.json()
  ),
  transports: [
    new winston.transports.File({
      filename: 'logs/error.log',
      level: 'error'
    }),
    new winston.transports.File({
      filename: 'logs/combined.log'
    }),
    new winston.transports.Console({
      format: winston.format.combine(
        winston.format.colorize(),
        winston.format.simple()
      )
    })
  ]
});

// Enable debug logging
logger.level = 'debug';

// Log with context
logger.debug('Reading point', {
  deviceId: 389001,
  pointId: 'IN1',
  protocol: 'bacnet'
});
```

Next Steps

- Performance Tuning
- REST API Debugging
- System Monitoring
- FAQ - Frequently asked questions